

Containerização II

Semana 3

CSDV-246 -DevOps



Conteúdo

- YAML
- Abordagens Imperativa e Declarativa
- Docker Compose
- Casos de Uso de Compose
- Compose File
- Compose CLI
- Administrar Múltiplos Contêineres
- Construção de Imagens
- Introdução ao Ansible



YAML

YAML



- Significa **Y**AML **A**in't **M**arkup **L**anguage.
- Formato de dados legível por humanos.
- Comumente usado para expressar configurações de aplicativos, serviços, etc..
- Usado por ferramentas como Kubernetes, Docker Compose, Ansible e outras.

YAML



XML

```
<Servers>
  <Server>
    <name>Server1</name>
    <owner>John</owner>
    <created>12232012</created>
    <status>active</status>
  </Server>
</Servers>
```

JSON

```
{
  Servers: [
    {
      name: Server1,
      owner: John,
      created: 12232012,
      status: active,
    }
  ]
}
```

YAML

```
Servers:
- name: Server1
  owner: John
  created: 12232012
  status: active
```

YAML



Estrutura Key/Value

Fruta: Laranja

Vegetal: Cenoura

Liquido: Agua

Carne: Frango



Estrutura de lista ordenada(Array)

Frutas:

Laranja

Banana

Tangerina

Vegetais:

Cenoura

Couve-flor

Salsa

Estrutura de lista ordenada (Array)

Frutas:

Laranja
Maçã
Banana



Frutas:

Laranja
Banana
Maçã



Estrutura Lista não ordenada (Dictionary)

Banana:

Calorias: 105

Gordura: 0.4g

Carboidratos: 27g

Uva:

Calorias: 62

Gordura: 0.3g

Carboidratos: 16g

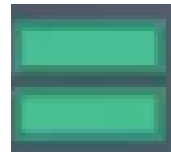
Estrutura Lista Classificada (Array)

Banana:

Calorias: 105

Gordura: 0.4g

Carboidratos: 27g



Banana:

Calorias: 105

Carboidratos: 27g

Gordura: 0.4g

Abordagens imperativas e declarativas

Abordagens imperativas e declarativas

- São dois paradigmas diferentes para gerenciar e definir a configuração e o comportamento dos sistemas.
- Eles são usados em várias áreas, como programação, gerenciamento de infraestrutura e DevOps.

Abordagens imperativas e declarativas

Abordagem imperativa

- Esse paradigma especifica exatamente o passo a passo, o conjunto de instruções que devem ser executadas para alcançar um resultado desejado.
- No que diz respeito ao nosso assunto, usamos esse paradigma quando executamos comandos, um após o outro, seguindo uma sequência específica, até que nossa configuração funcione conforme o esperado.
- Isso requer mais intervenção manual e repetitiva.

Abordagens Imperativas e Declarativas

Abordagem imperativa

- **Resultado desejado:** Site e banco de dados implantados usando contêineres Nginx e MySQL.

- **Solução:**

PASO 1: Baixar imagem do Nginx

```
docker pull nginx && docker pull mysql
```

PASO 2: Levantar o contêiner Nginx

```
docker run --name webserver -d -p 80:80 nginx
```

PASO 3: Levantar contêiner MySQL

```
docker run --name db -d -p 3306:3306 mysql
```

Abordagens Imperativas e Declarativas

Abordagem declarativa

- Consiste em instruir um programa sobre O QUE (Que solução queremos), em vez de dizer a ele COMO implementar a solução passo a passo.
- O usuário expressará o que espera obter por meio de uma linguagem específica do domínio ou DSL (por exemplo, YAML).

Abordagens Imperativas e Declarativas

Abordagem Declarativa

- **Resultado desejado:** Site e banco de dados implantados usando contêineres Nginx e MySQL.

Abordagens Imperativas e Declarativas

Abordagem Declarativa

- **Solução:** Descrever os contêineres a serem criados em um arquivo.

```
version: '3'
```

```
services:
```

```
  web:
```

```
    name: webserver
```

```
    image: nginx
```

```
    ports:
```

```
      - "80:80"
```

```
  db:
```

```
    name: db
```

```
    image: mysql
```

```
    ports:
```

```
      - "3306:3306"
```

Docker Compose

Docker Compose



- É uma ferramenta para definir e executar vários contêineres.
- Simplifica o gerenciamento de serviços, redes e volumes em um arquivo de configuração YAML simples e compactável.
- Esse arquivo YAML é chamado de **Compose File** e o **Compose CLI** interpreta o arquivo e envia as instruções para o Docker Engine.

Docker Compose



Beneficios

- Es más fácil administrar aplicaciones multi-contenedor.
- Fácil de leer y entender, lo que facilita la colaboración entre Desarrolladores, Operadores y Stakeholders.
- Se puede reutilizar el mismo Compose file en diferentes ambientes, aprovechando las variables personalizadas.

Casos de Uso de Compose

Casos de Uso de Compose



- Ambientes de desenvolvimento.
- Ambientes para testes automatizados.
- Implantações de host único em produção.
- Pipelines de Continuous Integration.

Compose File

Compose File



- Geralmente está localizado e com controle de versão no diretório raiz do código-fonte.
- A especificação do Compose é um padrão para definir aplicativos de vários contêineres independentes de plataforma ou ferramenta.



Compose File

- Este arquivo pode ser nomeado:
 - **compose.yaml** (preferido)
 - **compose.yml**
 - docker-compose.yaml
 - docker-compose.yml
 - Outros.
- O arquivo Compose tem a seguinte estrutura:
 - Version (Opcional)
 - Services (Necessário)
 - Networks
 - Volumes



Compose File

Versão e Nome

- A versão só é informativa se for adicionada ao arquivo de composição (opcional)
- O nome é o nome do projeto que será usado se não for especificado explicitamente.

```
version: "2"
```

```
name: myapp
```

```
services:
```

```
  foo:
```

```
    image: busybox
```

```
    command: echo "I'm running"
```

Compose File



Services

- Praticamente, eles são compostos por um contêiner ou um conjunto de contêineres.
- Eles permitem que você especifique como cada contêiner deve ser executado, qual imagem usar, as portas a serem expostas, as redes, volumes e outras configurações específicas de cada contêiner.

services:

foo:

image: busybox

command: echo "I'm running"

db:

image: mongo:latest

container_name: mongo

ports:

- "1234:27001"

environment:

- "PASSWORD=1234"

Compose File



Networks

- Eles permitem que os serviços se comuniquem entre si.
- Ele permite que você configure redes que podem ser reutilizadas em vários serviços.
- Se uma rede não estiver definida com um nome, os serviços serão adicionados a uma rede padrão.
- Para usar uma rede em vários serviços, o acesso a cada serviço deve ser concedido explicitamente usando o atributo `networks` em cada serviço.

services:

frontend:

image: example/webapp

container_name: frontend

networks:

- front-tier

- back-tier

networks:

front-tier:

back-tier:

Compose File



Volumes

- Cada serviço tem a capacidade de persistir os dados gerados.
- Permite configurar volumes que podem ser reutilizados em vários serviços.
- Também são válidos os *bind mounts*.
- Para usar um volume em vários serviços, você deve conceder explicitamente acesso a cada serviço usando o atributo volumes

services:

backend:

image: example/database

container_name: db

volumes:

- db-data:/etc/data

backup:

image: backup-services

container_name: backup

depends_on:

- backend

volumes:

- /data/backup:/var/lib/backup

volumes:

db-data:

Compose CLI

Compose CLI



- É a ferramenta que permite interagir com o Docker Compose através da linha de comando.
- Praticamente, ele é integrado ao Docker e é usado para executar, gerenciar, parar e excluir os serviços definidos no Compose File.
- O Compose File e a CLI automatizam o gerenciamento de uma pilha de serviços inteira.

Compose CLI



Criar e iniciar

- Comando:
docker compose **up**
docker compose **up** --opciones [NOMBRE DE SERVICIO]
- Exemplo:
Criar todos os serviços em segundo plano
docker compose **up** -d
Criar um serviço
docker compose **up** -d frontend

Compose CLI



Parar

- Comando:
docker compose **stop**
docker compose **stop** --Opções [NOMBRE DE SERVICIO]
- Exemplo:
Parar todos os serviços
docker compose **stop**
Parar um serviço
docker compose **stop** frontend

Compose CLI



Excluir

- Comando:
docker compose **rm**
docker compose **rm** --ópciones [NOME DE SERVICIO]
- Exemplo:
Excluir todos os serviços parados
docker compose **rm**
Excluir um serviço parado
docker compose **rm** frontend

Compose CLI



Pausar e retomar

- Comando:
docker compose **pause** [NOME DE SERVIÇO]
docker compose **unpause** [NOME DE SERVIÇO]
- Exemplo:
Pausar todos os serviços
docker compose **pause**
Retomar um serviço
docker compose **unpause** frontend

Gerenciar Múltiplos contêineres

Gerenciar Múltiplos contêineres



- Listar todos os serviços e seu status atual
docker compose **ps** -a
- Mostrar configurações do Compose file:
docker compose **config**
- Ver logs:
docker compose **logs**
- Acessar um serviço para executar comandos
docker compose **exec** frontend **bash**

Gerenciar Múltiplos Containers



Parar e excluir

- Comando (**cuidado!**):
docker compose **down**
docker compose **down** --opções [NOME DE SERVICIO]
- Exemplo:
Exclua todos os serviços e redes associados, mas sem excluir volumes
docker compose **down**
Excluir um serviço, redes e volume associado
docker compose **down** -v frontend

Gerenciar Múltiplos contêineres



- Examinar estatísticas de uso de recursos
docker compose **stats**
- Mostrar processos em execução
docker compose **top**
- Listar imagens de serviços:
docker compose **images**
- Criando imagens de serviço com o Dockerfile
docker compose **build** frontend

Construção de Imagens

Construção de Imagens



- Cada serviço também pode incluir uma seção de compilação, que define como criar a imagem do Docker para o serviço.
- A seção de construção é opcional.
- Se não for usada, a seção de construção será ignorada e o Compose File ainda é considerado válido.
- Recomenda-se revisar o [Compose Build Specification](#).

Construção de Imagens



- Para criar imagens:
 - No serviço, substitua o **"image"** por **"build"**.
 - Indicar o contexto de compilação se estiver em uma pasta diferente.
 - Indicar o Dockerfile se ele tiver um nome diferente.
 - Inserir a tag da nova imagem

services:

web:

build:

context: ./webapp/

dockerfile: web.Dockerfile

tags:

- web:v1.0

- web:latest

container_name: web

networks:

- web-tier

networks:

web-tier:

Construção de Imagens



Comandos

- Construir la imágen de un servicio sin crear un contenedor.
docker compose **build** web
- Construir la imágen de un servicio y luego levantar un contenedor:
docker compose **up** web
- Reconstruir la imagen y actualizar el contenedor:
docker compose **up --build** web

Construção de imagens



Vantagens e Desvantagens

Vantagens	Desvantagens
Automatiza o processo de criação de imagens	Não é ideal para implantações em ambientes de produção
Fácil atualização de imagem ao alterar o Dockerfile	Reconstruções desnecessárias se o cache não for gerenciado corretamente
Configura ambientes de desenvolvimento.	Pode aumentar o tempo de elevação do contêiner se as imagens forem grandes

Construção de Imagens



Alguns casos de uso

Caso de uso	Descrição
Desenvolvimento local	Personalize imagens de aplicativos de desenvolvimento com dependências e configurações específicas.
Automatizando o ambiente de desenvolvimento	Configure ambientes de desenvolvimento completos com vários serviços para toda a equipe de maneira uniforme.
Testes automatizados (CI/CD)	Criação de imagens para ambientes de teste automatizados antes de executar testes.

Introdução ao Ansible

Introdução ao Ansible



- Ansible é uma ferramenta de automação de código aberto.
- Use scripts chamados "Playbooks", são scripts fáceis de ler e entender porque são baseados na sintaxe YAML.
- O Playbook declara o estado desejado de um sistema local ou remoto e o Ansible garante que o sistema permaneça nesse estado.
- É amplamente utilizado em DevOps para automatizar o provisionamento de servidores, configuração de rede, implantação de software, etc.

Introdução ao Ansible



Características Principais:

- **Agentless:** Você não precisa de um software especial (agente) para ser instalado nos sistemas que está gerenciando.
- **Comunicação:** Comunica-se diretamente com servidores via SSH ou WinRM (para Windows).
- **Declarativo:** Permite definir o estado desejado da infraestrutura em vez de especificar as etapas para chegar a esse estado.
- **Idempotente:** Quando o sistema está no estado descrito pelo Playbook, o Ansible não altera nada, mesmo que o Playbook seja executado várias vezes.

Introdução ao Ansible



Casos de Uso

- É uma das ferramentas mais utilizadas no Gerenciamento de Configuração.
- Automatize a implantação de aplicativos em um ou vários servidores ao mesmo tempo.
- Você pode coordenar várias tarefas entre diferentes servidores ao mesmo tempo.
- Ele provisiona servidores recém-criados com todo o software e dependências necessárias para que nosso aplicativo funcione quando o implantamos.

Introdução ao Ansible



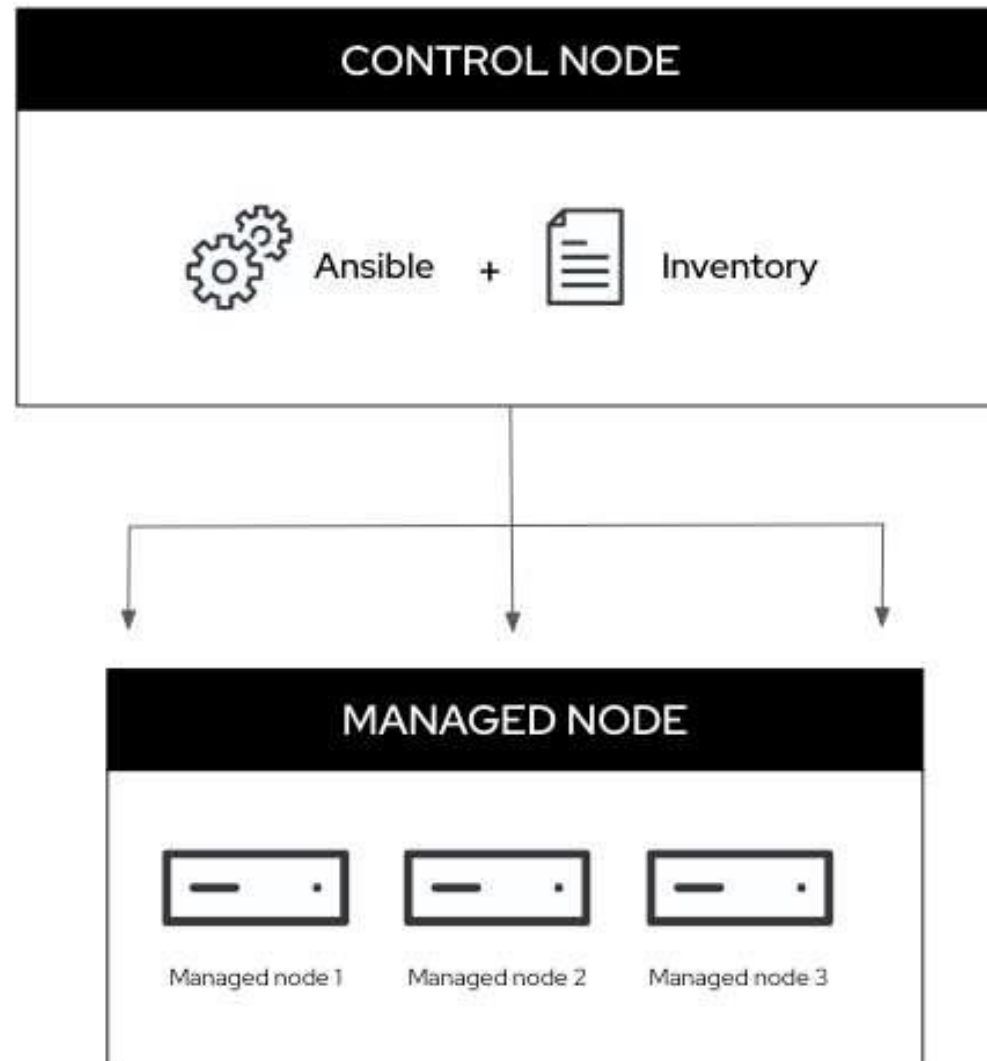
Componentes:

- **Control Node:** Basicamente, é a máquina a partir da qual os diferentes comandos e ferramentas fornecidos pela CLI do Ansible são executados.
- **Managed Node:** Eles são os dispositivos de destino, como servidores, dispositivos de rede ou qualquer computador. Eles também são conhecidos como "hospedeiros". Você não precisa instalar o Ansible nesses nós gerenciados.

Introdução ao Ansible



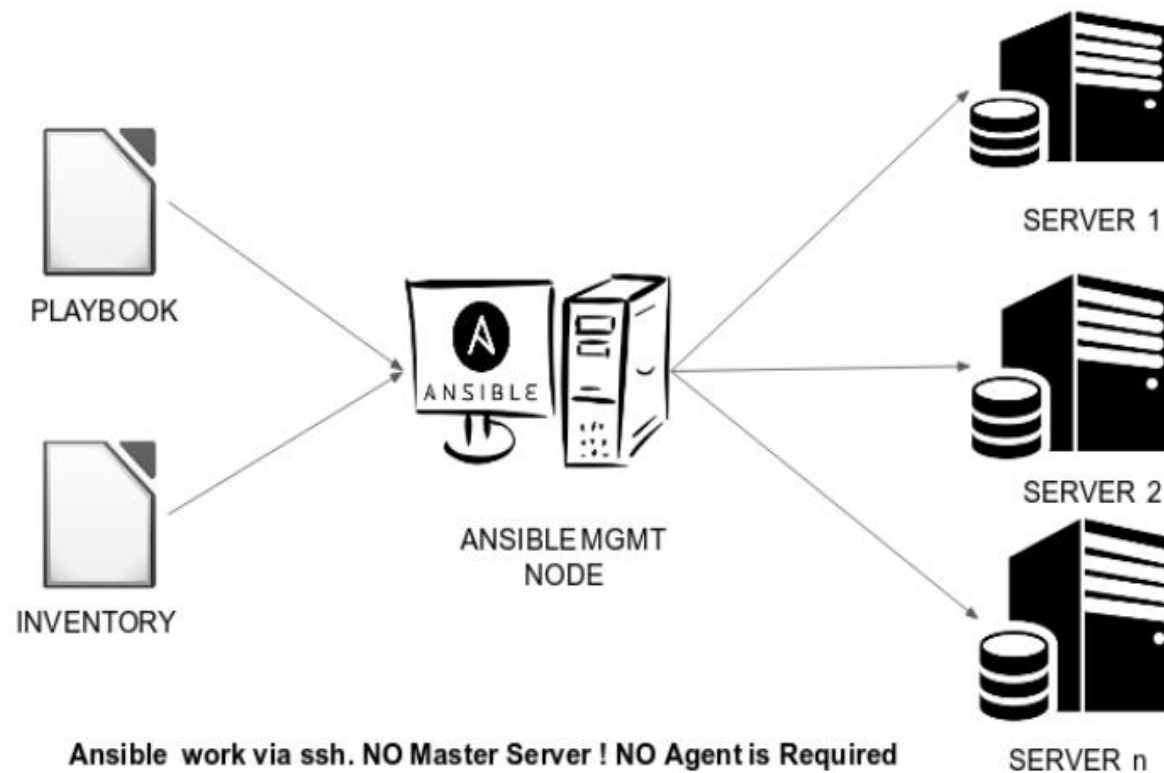
Arquitetura do Ansible



Introdução ao Ansible



Ansible - how it works



Introdução ao Ansible



Componentes:

- **Playbooks:** Esses são arquivos de configuração no formato YAML que descrevem as tarefas que o Ansible deve executar em sistemas remotos.
- **Inventory:** Esta é a lista de servidores ou nós (chamados de hosts) que o Ansible gerenciará.
- **Modules:** Esses são pedaços de código que o Ansible pode executar nos nós, como instalar software, configurar arquivos, manipular serviços, executar comandos do sistema e assim por diante.
- **Roles:** Agrupe tarefas relacionadas para simplificar a reutilização e organização de tarefas.

Introdução ao Ansible



Playbook

- Para entender em detalhes o que é um Playbook, primeiro devemos entender os seguintes conceitos:
 - **Task:** É uma "ação", uma "operação" que será executada e aplicada em cada nó gerenciado.
 - **Role:** É uma maneira estruturada de organizar o conteúdo do Ansible, como tarefas, modelos, arquivos e variáveis, em um formato reutilizável.
 - **Play:** Uma lista ordenada de tarefas, variáveis e funções atribuídas a nós gerenciados. Essas tarefas podem ser executadas repetidamente.

Introdução ao Ansible

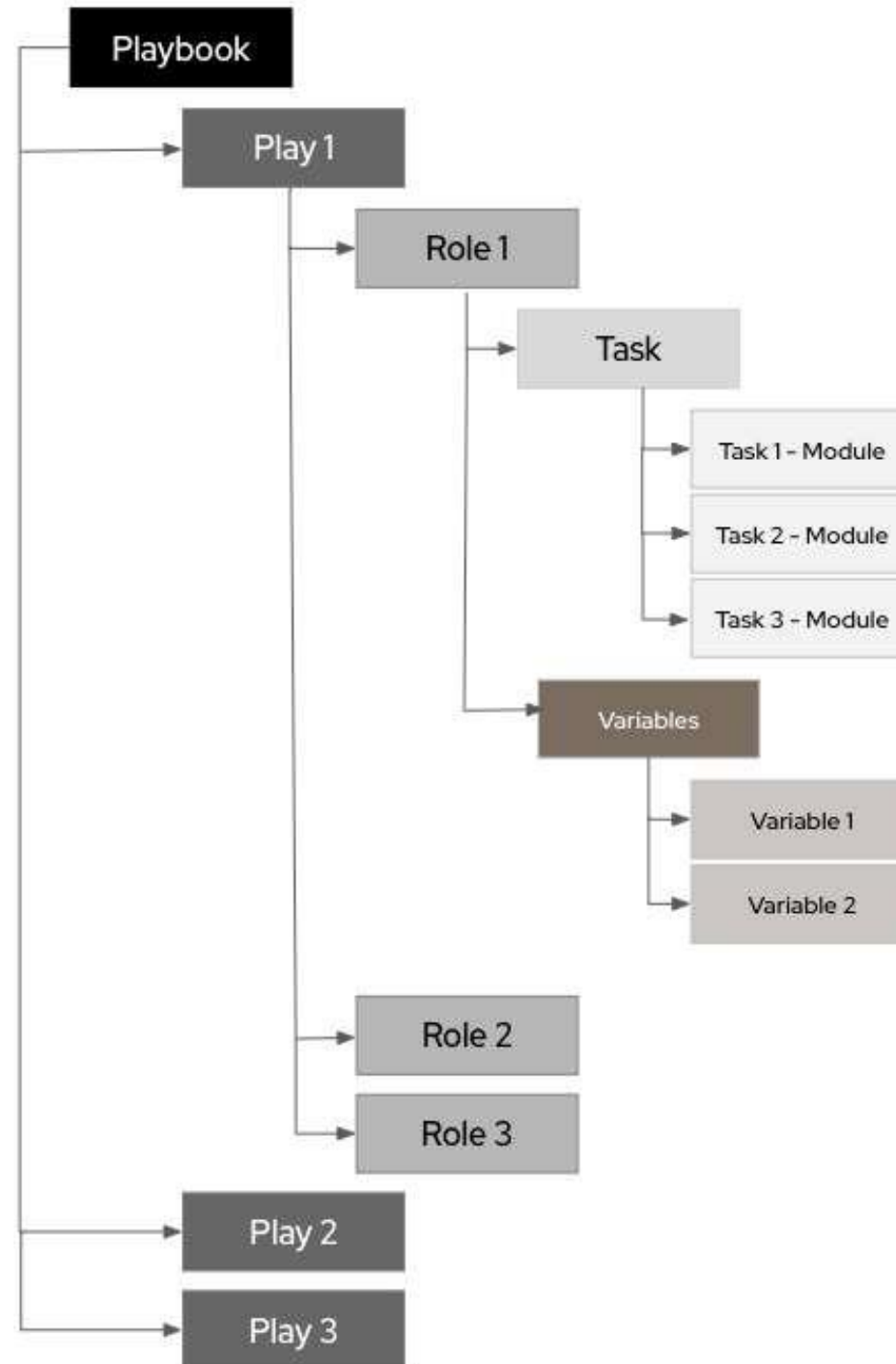


Playbook

- Agora, um Playbook é uma lista de tarefas ou instruções que são executadas automaticamente em um ou mais grupos de hosts específicos (nós gerenciados).
- Você pode combinar uma ou mais tarefas e criar um "play".
- Um Playbook pode incluir uma ou mais *Play* que são exibidas em uma lista ordenada e executadas sequencialmente (de cima para baixo).

Introdução ao Ansible

Estrutura do Playbook



Introdução ao Ansible

Exemplo de playbook



```
---
- name: Update web servers
  hosts: webservers
  remote_user: root

  tasks:
    - name: Ensure apache is at the latest version
      ansible.builtin.yum:
        name: httpd
        state: latest

    - name: Write the apache config file
      ansible.builtin.template:
        src: /srv/httpd.j2
        dest: /etc/httpd.conf

- name: Update db servers
  hosts: databases
  remote_user: root

  tasks:
    - name: Ensure postgresql is at the latest version
      ansible.builtin.yum:
        name: postgresql
        state: latest

    - name: Ensure that postgresql is started
      ansible.builtin.service:
        name: postgresql
        state: started
```

Introdução ao Ansible



Inventory

- Um inventário contém informações sobre os nós gerenciados que você deseja que o Ansible organize.
- Os inventários podem agrupar nós, permitindo que você execute Playbooks simultaneamente em vários hosts.
- Os inventários são normalmente criados no formato YAML ou INI.
- O local padrão para este arquivo é [/etc/ansible/hosts](#)
- Você pode especificar seu próprio arquivo de inventário em um caminho diferente.

Introdução ao Ansible



Inventário no formato YAML

```
ungrouped:
  hosts:
    mail.example.com:
webservers:
  hosts:
    foo.example.com:
    bar.example.com:
dbservers:
  hosts:
    one.example.com:
    two.example.com:
    three.example.com:
```

Inventário no formato INI

```
mail.example.com

[webservers]
foo.example.com
bar.example.com

[dbservers]
one.example.com
two.example.com
three.example.com
```

Introdução ao Ansible



Executar Playbooks

- Para executar uma Playbook, usa-se o comando `ansible-playbook`.
- Exemplo:
`ansible-playbook playbook.yml --inventory servers-list.yml`
- Para ver uma saída mais detalhada enquanto o Playbook está em execução, você deve usar o comando `--verbose`. Isso pode ajudar na depuração.
- Exemplo:
`ansible-playbook playbook.yml --verbose`

Introdução ao Ansible



Executar Playbooks em Modo de verificação

- O Modo de Verificação permite que você execute um Playbook sem aplicar alterações aos nós gerenciados.
- O modo de verificação pode ser usado para testar guias estratégicos antes de executá-los em um ambiente de produção.
- Basta adicionar o `--check` o `-C` ao comando `ansible-playbook`.
- Exemplo:
`ansible-playbook --check playbook.yml`

Introdução ao Ansible



Verificação de Playbooks

- O Ansible fornece uma ferramenta para verificar os Playbooks em busca de falhas de sintaxe e outros problemas antes de executá-los.
- O modo de verificação pode ser usado para testar guias estratégicos antes de executá-los em um ambiente de produção..
- Adicione as opções `--check`, `--diff`, `--list-hosts`, `--list-tasks`, y `--syntax-check` ao comando `ansible-playbook`.
- Você também pode usar o comando `ansible-lint`

Introdução ao Ansible



Linting de Playbooks

- ansible-lint fornece feedback detalhado sobre Playbooks antes de executá-los.
- Exemplo:
ansible-lint verify-apache.yml
[403] Package installs should not use latest
verify-apache.yml:8
Task/Handler: ensure apache is at the latest version
- Revisar regras de linting: [Ansible Lint Rules](#).

Q&A